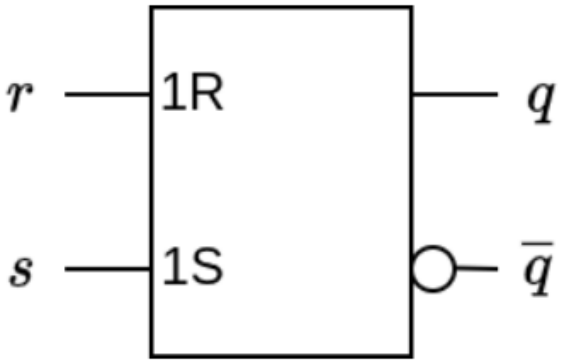
05 Schaltwerke

- Schaltnetze operieren kombinatorisch, d.h. der Ausgang der Schaltung hängt ausschließlich von den aktuellen Eingängen ab
- Heute betrachten wir Schaltwerke, d.h. Schaltungen mit einem Gedächtnis, die nicht nur von den aktuellen Werten der Eingänge abhänge

1. **Multiplexer** wählen eines von mehreren Signalen aus.
2. **Addierer** rechnen, und aus ihnen entsteht auch der **Subtrahierer**.
3. **Komparator** und **Barrel-Shifter** vergleichen und verschieben.
4. Eine **ALU** ist all das zusammen, mit einem Multiplexer als Auswahlschalter.

Asynchrone Speicherelemente

RS-Latch



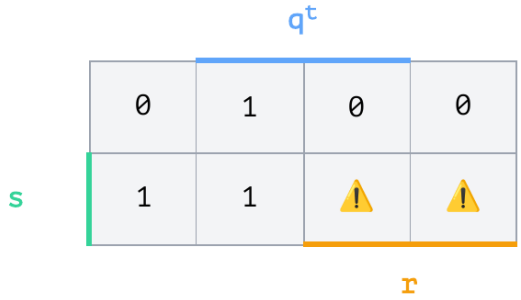
![[05_sequential.pdf#page=9&rect=383,101,635,314|05_sequential, p.9]

r	s	q(t+1)	Verhalten
0	0	q(t)	**Speichern**
0	1	1	Setzen
1	0	0	Rücksetzen
1	1		verboten

Asynchrones RS-Latch: Herleitung per KV-Diagramm

Wahrheitstafel des RS-Latches

r	s	q^t	q^{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	⚠
1	1	1	⚠

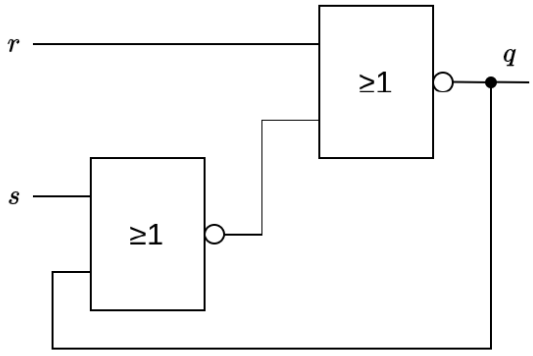


$$q^{t+1} = (\overline{r} \wedge s) \vee (\overline{r} \wedge q^t)$$

Schaltung aus NOR Gattern

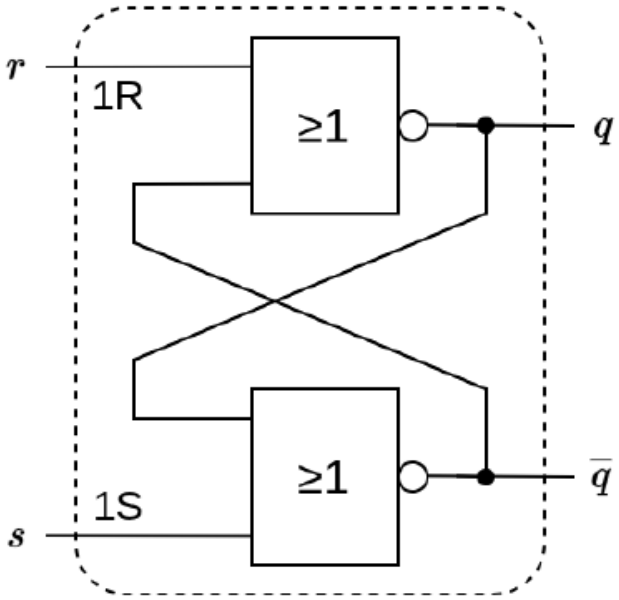
- Umformen zu einer NOR-only Lösung:

$$\begin{aligned} q^{t+1} &= (\overline{r} \wedge s) \vee (\overline{r} \wedge q^t) \\ &= \overline{r} \wedge (s \vee q^t) && \text{(D1)} \\ &= \overline{\overline{\overline{r} \wedge (s \vee q^t)}} && \text{(Doppel-Kompl.)} \\ &= \overline{r \vee \overline{s \vee q^t}} && \text{(De Morgan)} \end{aligned}$$



RS-Latch mit NOR-Gattern

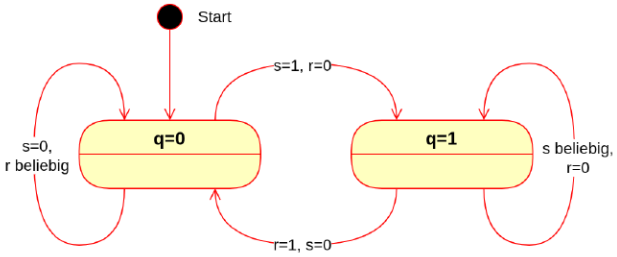
⇒



Gängige Darstellung des RS-Latches

Zustandsdiagramm

- **Zustände** ($q = 0, q = 1$) als Kreise / abgerundete Rechtecke
- **Übergänge** (Transitionen) als Pfeile zwischen Zuständen
- Eingänge als Trigger / Event ($s = 1, r = 0,$ etc.) für Transitionen

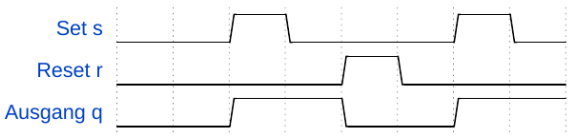


Zustandsdiagramm des RS-Latches

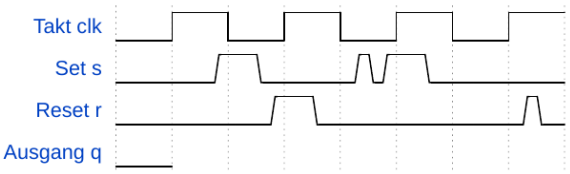
- Bei rs=11
- beide NOR-Ausgänge auf 0: (Komplementärbedingung verletzt)
- Danach gleichzeitig : beide NOR-Gatter schalten auf 1, dann wieder auf 0, ...
- Ein Wettlauf der Gatterlaufzeiten (Race Condition) entscheidet über den Folgezustand

Taktgesteuerte Speicherelemente

- Ein gemeinsamer Takt bestimmt, wann Speicherelemente ihren Zustand wechseln dürfen.



Asynchrones RS-Latch



Synchrones RS-Latch

alles was nicht im gelben passwier wird ignoriert

D-Latch

(delay)

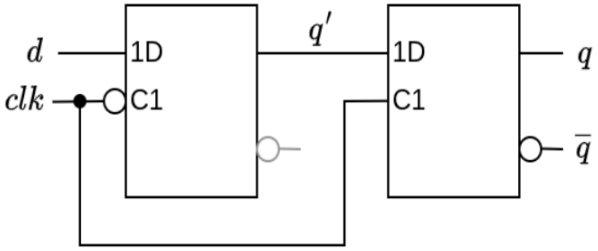
clk	d	q(t+1)
0	0	q(t)
0	1	q(t)
1	0	0
1	1	1

Das **D-Latch** ist ein RS-Latch mit $\mathbf{s = d}$ und $\mathbf{r = \neg d}$ (ein Inverter vor r). Es hat dadurch nur einen Dateneingang d. Die verbotene Kombination rs=11 kann **schaltungstechnisch nie auftreten**.

Verhalten: Bei clk=1 ist das Latch **durchsichtig**, q folgt d. Bei clk=0 hält es.

FlipFlop

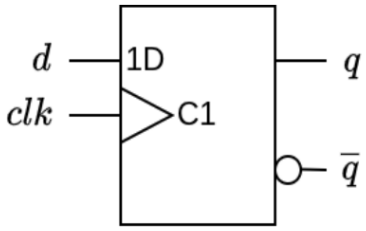
- Zwei D-Latches (1DC1) hintereinandergeschaltet
  - *Leader* erhält invertierten Takt
  - *Follower* erhält normalen Takt
- Bei $clk = 0$
  - Leader übernimmt d, der Follower sperrt
- Bei $clk = 1$
  - Leader sperrt, Follower übernimmt dessen Wert q' auf den Ausgang q



Aufbau eines D-Flipflops aus D-Latches

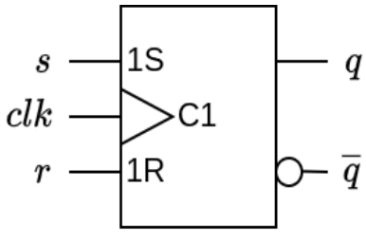
Weitere Varianten

D-Flipflop (*Delay*)



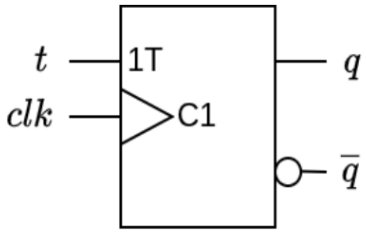
clk	d	q^{t+1}
0 / 1 / ↓	-	q^t
↑	0	0
↑	1	1

RS-Flipflop (*Set/Reset*)



clk	s	r	q^{t+1}
0 / 1 / ↓	-	-	q^t
↑	0	0	q^t
↑	0	1	0
↑	1	0	1
↑	1	1	⚠️

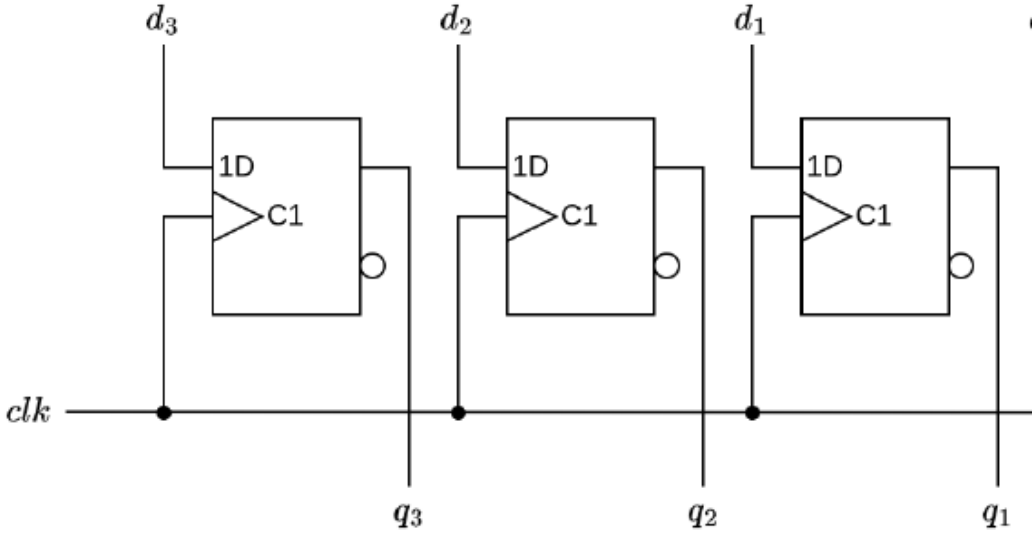
T-Flipflop (*Toggle*)



clk	t	q^{t+1}
0 / 1 / ↓	-	q^t
↑	0	q^t
↑	1	$\overline{q^t}$

Register

- Die ALU ist ein Schaltnetz und kann nichts speichern. Ein Flipflop speichert nur 1 Bit. Für ein n-Bit-Datenwort braucht man n Flipflops, und das nennt man **Register**. Schaltnetze rechnen, Register merken.



Register aus D-Flipflops

Auffangregister

- Es fängt mit einer Taktflanke ein ganzes Datenwort auf und hält es fest.
- Ein Enable-Eingang (e)ermöglicht zwei Anwendungsszenarien:
  - bei e=1 wird das Register parallel geladen
  - bei e=0 behält das Register seinen Wert
- Anwendung für Auffangregister in Rechner:
  - Pufferregister für Operanden und Zwischenergebnisse von Berechnungen